

Лабораторна робота №2

Тема: Багатошаровий перцептрон для розпізнавання рукописних цифр бібліотеки MNIST та графічних символів бібліотеки Fashion MNIST

Мета роботи: написати комп'ютерну програму мовою Python, що створює та навчає нейронну мережу для розпізнавання рукописних цифр та графічних символів.

Вхідні дані:

- Кількість шарів нейронної мережі: 3.
- Вхідні дані для нейронної мережі: зображення розміром 28×28 пікселів.
- Середовище програмування: Python 3.6 або вище.
- Бібліотека, що використовується: NumPy.

Методичні вказівки до лабораторної роботи

1) Організація середовища розробки.

Встановити Python. Систему програмування мовою Python 3.6 або вище для Windows можна завантажити з офіційного сайту <https://www.python.org/downloads/windows/> (рекомендується використовувати Windows x86 executable installer). Перед встановленням необхідно вибрати "Add Python to PATH".

2) Встановлення бібліотеки NumPy

1. Запустіть командний рядок (для цього наберіть cmd у вікні пошуку панелі завдань Windows).

2. Запустіть команду для інсталяції пакету:

```
pip3 install numpy
```

3) Встановлення робочої папки проекту

Створіть каталог NeuralNetwork, у якому Ви зберігатимете вхідні тексти програм, створюваних під час виконання практичних завдань (наприклад, C:\NeuralNetwork). У каталозі NeuralNetwork створіть підкаталог Network1, де зберігатимуться вихідні коди завдання 1.

4) Створення нейронної мережі

Запустіть середовище розробки (для запуску середовища розробки IDLE наберіть idle у вікні пошуку панелі завдань Windows). Створіть новий файл програми (меню File/New File). Збережіть цей файл у каталозі Network1 під ім'ям network (меню File/Save).

Розширення .py буде встановлено за замовчуванням.

Скопіюйте у вікно програми network.py наступні команди та впишіть свої дані:

```

#####
network.py
Модуль створення та навчання ШНМ для розпізнавання рукописних цифр з
використанням методу градієнтного спуску.
Група:<Вказати номер групи>
ПІБ:<Вказати ПІБ студента>
#####

#### Бібліотеки
# Стандартні бібліотеки
import random # бібліотека функцій для генерації випадкових значень

# Сторонні бібліотеки
import numpy as np # бібліотека функцій для роботи з матрицями

""" ---Розділ описів--- """
""" --Опис класу Network--"""
class Network(object): # використовується для опису нейронної мережі
    def __init__(self, sizes): # конструктор класу
        # self - покажчик на об'єкт класу
        # sizes - список розмірів шарів ШНМ
        self.num_layers = len(sizes) # задаємо кількість шарів
        self.sizes = sizes # задаємо список розмірів шарів
        self.biases = [np.random.randn(y, 1) for y in sizes[1:]] # задаємо
випадкові початкові зміщення
        self.weights = [np.random.randn(y, x) for x, y in zip(sizes[:-1],
sizes[1:])] # задаємо випадкові початкові ваги зв'язків
""" --Кінець опису класу Network--"""
--- Кінець розділу описів--- """

""" ---Тіло програми--- """
net = Network([2, 3, 1]) # створюємо ШНМ з 3-х шарів
""" ---Кінець тіла програми--- """

""" Вивід результату на екран: """
print('Мережа net:')
print('Кількість шарів:', net.num_layers)
for i in range(net.num_layers):
    print('Кількість нейронів у шарі', i, ':', net.sizes[i])
for i in range(net.num_layers-1):
    print('W ', i+1, ':')
    print(np.round(net.weights[i], 2))
    print('b ', i+1, ':')
    print(np.round(net.biases[i], 2))

```

Збережіть файл `network.py` та виконайте програму `network`. Щоб запустити виконання програми, виберіть `Run/Run Module` (або натисніть `F5`). В результаті буде створено об'єкт класу `Network`, який задає трирівневу нейронну мережу з відповідними параметрами. При створенні об'єкта класу `Network` ваги та зміщення ініціалізуються випадковим чином. Для ініціалізації цих величин використовується функція `np.random.randn` із бібліотеки `NumPy`. Ця функція генерує числа з нормальним розподілом для масиву заданої розмірності.

Визначення сигмоїдальної функції

У якості функції активації для нейронів мережі використовується сигмоїдальна функція, що обчислює вихідний сигнал штучного нейрона. Нижче наведено код, для визначення функції. Додайте цей код до розділу описів програми `network.py`.

```
def sigmoid(z): # визначення сигмоїдної функції активації
    return 1.0/(1.0+np.exp(-z))
```

Зверніть увагу, що для опису сигмоїдальної функції використовується функція для обчислення експоненти з бібліотеки `NumPy`, це дозволяє передавати масив як вхідний параметр сигмоїдальної функції. У цьому випадку функція експоненти застосовується поелементно, тобто у векторизованій формі.

Метод `feedforward`

Додайте метод `feedforward` в опис класу `Network`.

```
def feedforward(self, a):
    for b, w in zip(self.biases, self.weights):
        a = sigmoid(np.dot(w, a)+b)
    return a
```

Цей метод здійснює підрахунок вихідних сигналів нейронної мережі при заданих вхідних сигналах. Параметр $\mathbf{a}$ є масивом $n \times 1$, де n – кількість нейронів вхідного шару. Функція `np.dot` обчислює добуток матриць. Для підрахунку вихідних значень нейронної мережі необхідно один раз викликати метод `feedforward`, в результаті чого вихідні сигнали будуть послідовно обчислені для всіх шарів нейронної мережі.

Навчання нейронної мережі

Для реалізації механізму навчання нейронної мережі додамо метод SGD, який реалізує *стохастичний градієнтний спуск*. Метод має такі параметри:

«`Training_data`» – навчальна вибірка, яка складається з пар виду $(\vec{x}, \vec{y})$, де $\vec{x}$ – вектор вхідних сигналів, а $\vec{y}$ – очікуваний вектор вихідних сигналів;

«`epochs`» – кількість епох навчання;

«`mini_batch_size`» - розмір підвиборки;

«`eta`» - швидкість навчання;

«`test_data`» - (необов'язковий параметр); якщо даний аргумент не пустий, то програма після кожної епохи навчання здійснює оцінку роботи мережі та показує досягнутий прогрес.

Додайте програмний код метода SGD в розділ в опису класу `Network`:

```

def SGD( # Стохастичний градієнтний спуск
        self # покажчик на об'єкт класу
        , training_data # навчальна вибірка
        , epochs # кількість епох навчання
        , mini_batch_size # розмір підвиборки
        , eta # швидкість навчання
        , test_data # тестова вибірка
    ):
        test_data = list(test_data) # створюємо список об'єктів тестової
        вибірки
        n_test = len(test_data) # обчислюємо довжину тестової вибірки
        training_data = list(training_data) # створюємо список об'єктів
        навчальної вибірки
        n = len(training_data) # обчислюємо розмір навчальної вибірки
        for j in range(epochs): # цикл по епохам
            random.shuffle(training_data) # змішуємо елементи навчальної
            вибірки
            mini_batches = [training_data[k:k+mini_batch_size] for k in
            range(0, n, mini_batch_size)] # створюємо підвибірки
            for mini_batch in mini_batches: # цикл по підвибіркам
                self.update_mini_batch(mini_batch, eta) # один крок
                градієнтного спуску
            print ("Epoch {0}: {1} / {2}".format(j,
            self.evaluate(test_data), n_test)) # дивимось прогрес у навчанні

```

Цей програмний код працює наступним чином. На початку кожної епохи навчання елементи навчальної вибірки перемішуються (переставляються у випадковому порядку) за допомогою функції `shuffle()` з бібліотеки `random`, після чого навчальна вибірка послідовно розбивається на підвибірки довжини `mini_batch_size`. Для кожної вибірки виконується один крок градієнтного спуску за допомогою методу `update_mini_batch` (див. нижче). Після того, як виконано останній крок градієнтного спуску, тобто, виконано метод `update_mini_batch` для останньої підвибірки, на екран виводиться досягнутий прогрес у навчанні нейронної мережі, що обчислюється на тестовій вибірці за допомогою методу `evaluate` (див. нижче).

Аналізуючи програмний код методу `update_mini_batch`, можна побачити, що основна частина обчислень здійснюється при виклику методу `backprop` (див. нижче). Цей метод класу `Network` реалізує алгоритм *зворотного поширення помилки*, що є швидким способом обчислення градієнта *функції вартості*. Таким чином, метод `update_mini_batch` обчислює градієнти для кожного прецеденту (x^i , y^i) у підвибірці, а потім відповідним чином оновлює ваги та зміщення нейронної мережі. Додайте код методу `update_mini_batch` до розділу в описах класу `Network`:

```

def update_mini_batch( # крок градієнтного спуску
    self                # покажчик на об'єкт класу
    , mini_batch        # підвибірка
    , eta               # швидкість навчання
):
    nabla_b = [np.zeros(b.shape) for b in self.biases] # список
    градієнтів dC/db для кожного шару (спочатку заповнюється нулями)
    nabla_w = [np.zeros(w.shape) for w in self.weights] # список
    градієнтів dC/dw для кожного шару (спочатку заповнюється нулями)
    for x, y in mini_batch:
        delta_nabla_b, delta_nabla_w = self.backprop(x, y) # пошарово
        обчислюємо градієнти dC/db та dC/dw для поточного прикладу (x, y)
        nabla_b = [nb+dnb for nb, dnb in zip(nabla_b, delta_nabla_b)] #
        підсумовуємо градієнти dC/db для різних прикладів поточної підвибірки
        nabla_w = [nw+dnw for nw, dnw in zip(nabla_w, delta_nabla_w)] #
        підсумовуємо градієнти dC/dw для різних прикладів поточної підвибірки
        self.weights = [w-(eta/len(mini_batch))*nw
                        for w, nw in zip(self.weights, nabla_w)] #
        оновлюємо всі ваги w нейронної мережі
        self.biases = [b-(eta/len(mini_batch))*nb
                       for b, nb in zip(self.biases, nabla_b)] # оновлюємо
        всі зміщення b нейронної мережі

```

Скопіюйте у розділ опису класу Network програмний код методу backprop, що реалізує алгоритм зворотного розповсюдження:

```

def backprop( # Алгоритм зворотного розповсюдження
    self      # покажчик на об'єкт класу
    , x       # вектор вхідних сигналів
    , y       # очікуваний вектор вихідних сигналів
):
    nabla_b = [np.zeros(b.shape) for b in self.biases] # список
    градієнтів dC/db для кожного шару (спочатку заповнюється нулями)
    nabla_w = [np.zeros(w.shape) for w in self.weights] # список
    градієнтів dC/dw для кожного шару (спочатку заповнюється нулями)

    # визначення змінних
    activation = x # вихідні сигнали шару (спочатку відповідають вихідним
    сигналам 1-го шару або вхідним сигналам мережі)
    activations = [x] # список вихідних сигналів по всіх шарах
    (спочатку містить тільки вихідні сигнали 1-го шару)
    zs = [] # список активаційних потенціалів по всіх шарах
    (спочатку пустий)

    # пряме розповсюдження
    for b, w in zip(self.biases, self.weights):
        z = np.dot(w, activation)+b # зчитуємо активаційні потенціали
        поточного шару
        zs.append(z) # додаємо елемент (активаційні потенціали шару) в
        кінець списку
        activation = sigmoid(z) # зчитуємо вихідні сигнали поточного
        шару, застосовуючи сигмоїдальну функцію активації до активаційних
        потенціалів шару
        activations.append(activation) # додаємо елемент (вихідні сигнали
        шару) в кінець списку

```

```

        # зворотне розповсюдження
        delta = self.cost_derivative(activations[-1], y) *
sigmoid_prime(zs[-1]) # зчитуємо міру впливу нейронів вихідного шару L на
величину помилки (BP1)
        nabla_b[-1] = delta # градієнт dC/db для шару L (BP3)
        nabla_w[-1] = np.dot(delta, activations[-2].transpose()) # градієнт
dC/dw для шару L (BP4)

        for l in range(2, self.num_layers):
            z = zs[-l] # активаційні потенціали l-го шару (рухаємось по
списку справа наліво)
            sp = sigmoid_prime(z) # зчитуємо сигмоїдальну функцію від
активаційних потенціалів l-го шару
            delta = np.dot(self.weights[-l+1].transpose(), delta) * sp #
зчитуємо міру впливу нейронів l-го шару на величину помилки (BP2)
            nabla_b[-l] = delta # градієнт dC/db для l-го шару (BP3)
            nabla_w[-l] = np.dot(delta, activations[-l-1].transpose()) #
градієнт dC/dw для l-го шару (BP4)
        return (nabla_b, nabla_w)

```

Скопіюйте у розділ опису класу `Network` програмний код методу `evaluate`, що демонструє прогрес у навчанні:

```

def evaluate(self, test_data): # Оцінка прогресу в навчанні
    test_results = [(np.argmax(self.feedforward(x)), y)
                     for (x, y) in test_data]
    return sum(int(x == y) for (x, y) in test_results)

```

Зазначений метод повертає кількість прецедентів тестової вибірки, для яких нейронна мережа видає правильний результат. Тестова вибірка складається з пар (x, y) , де x – вектор розмірності 784, що містить зображення цифри, а y – ціле числове значення цифри, яка зображена на картинці. Відповідь нейронної мережі визначається як номер нейрона у вихідному шарі, що має найбільше значення функції активації. Метод `evaluate` викликається у методі `SGD` після завершення чергової епохи навчання.

Скопіюйте у розділ опису класу `Network` програмний код методу `cost_derivative`, який обчислює вектор приватних похідних $\nabla C(\vec{a}^L) = \vec{a}^L - \vec{y}$:

```

def cost_derivative(self, output_activations, y): # Обчислення приватних
похідних функції вартості по вихідним сигналам останнього шару
    return (output_activations-y)

```

Зазначений метод викликається у методі `backprop`.

Скопіюйте в кінець розділу описів (після функції `sigmoid`) код функції `sigmoid_prime`, що обчислює похідну сигмоїдальної функції:

```

def sigmoid_prime(z): # Похідна сигмоїдальної функції
    return sigmoid(z) * (1-sigmoid(z))

```

Збережіть та закрийте файл `network.py`.

Робота з базою даних MNIST (Modified National Institute of Standards and Technology)

Для навчання нейронної мережі ми можемо використати архів `mnist.pkl.gz` із сайту <https://www-labs.iro.umontreal.ca/~lisa/deep/data/mnist/> Лабораторії машинного навчання Університету Монреалю, сформований на основі бази даних MNIST, який містить 70 000 зображень рукописних цифр, розділених на три набори *(але ми можемо використати й інші методи роботи з цією бібліотекою, розберіться самостійно, як саме це можна зробити та вкажіть це у висновках або використовуйте у своїй програмі)*:

- 1) `training_data` – набір із 50 000 зображень призначений для навчання нейронних мереж;
- 2) `validation_data` – набір із 10 000 зображень призначений для поточної оцінки роботи алгоритму навчання та підбору параметрів навчання (використовується в наступних лабораторних роботах);
- 3) `test_data` – набір із 10 000 зображень призначений для перевірки роботи нейронних мереж.

Кожен набір складається з двох списків: списку зображень (у градаціях сірого) та відповідного списку цифр у діапазоні від 0 до 9. Зображення представлене у вигляді одномірного `numpy`-масиву розміру $784 = 28 \times 28$ значень від 0 до 1, де 0 відповідає чорному кольору пікселя, а 1 – білому.

Завантажте архів з сайту <https://www-labs.iro.umontreal.ca/~lisa/deep/data/mnist/> та збережіть у директорії `Network1`.

Функції для роботи з базою даних MNIST доцільніше винести в окремий файл. Створіть новий файл `mnist_loader` та збережіть його у директорії `Network1`. Скопіюйте у вікно програми `mnist_loader.py` наступні команди та впишіть свої дані:

```
"""
mnist_loader.py
~~~~~
Модуль для підключення та використання бази даних MNIST.

Група: [Вказати номер групи]
ПІБ: [Вказати ПІБ студента]
"""
import gzip # бібліотека для стиснення та розпакування файлів gzip та
gunzip.
import pickle # бібліотека для збереження та завантаження складних
об'єктів Python.
import numpy as np # бібліотека для роботи з матрицями

def load_data():

    f = gzip.open('mnist.pkl.gz', 'rb') # відкриваємо стиснений файл gzip у
    двійковому режимі
    training_data, validation_data, test_data = pickle.load(f,
    encoding='latin1') # завантажуюмо таблиці з файлу
    f.close() # закриваємо файл
    return (training_data, validation_data, test_data)
```

Для використання бази даних MNIST у нашій програмі необхідно скоригувати формати наборів `training_data`, `validation_data` та `test_data`. Це робиться у функції `load_data_wrapper`. Скопіюйте у файл `mnist_loader` наступний програмний код.

```
def load_data_wrapper():
    tr_d, va_d, te_d = load_data() # ініціалізація наборів даних у форматі MNIST
    training_inputs = [np.reshape(x, (784, 1)) for x in tr_d[0]] #
    перетворення масивів розміру 1 на 784 до масивів розміру 784 на 1
    training_results = [vectorized_result(y) for y in tr_d[1]] #
    представлення цифр від 0 до 9 у вигляді масивів розміру 10 на 1
    training_data = zip(training_inputs, training_results) # формуємо
    набір навчальних даних з пар (x, y)
    validation_inputs = [np.reshape(x, (784, 1)) for x in va_d[0]] #
    перетворення масивів розміру 1 на 784 до масивів розміру 784 на 1
    validation_data = zip(validation_inputs, va_d[1]) # формуємо
    набір даних для перевірки з пар (x, y)
    test_inputs = [np.reshape(x, (784, 1)) for x in te_d[0]] #
    перетворення масивів розміру 1 на 784 до масивів розміру 784 на 1
    test_data = zip(test_inputs, te_d[1]) # формуємо набір тестових даних
    з пар (x, y)
    return (training_data, validation_data, test_data)
```

Ця функція перетворює `training_data` на список, що містить 50 000 пар (x, y) , де x є 784-вимірним `numpy`-масивом, що містить вхідне зображення, а y – це 10-мірний `numpy`-масив, що є вектором, у якого координата з порядковим номером, що відповідає цифрі на зображенні, дорівнює одиниці, а інші координати нульові. Аналогічні перетворення робляться для наборів `validation_data` та `test_data`.

Для перетворення числа у вектор-стовпець (10-мірний `numpy` масив), використовується наступна функція `vectorized_result`. Скопіюйте її програмний код у файл `mnist_loader`.

```
def vectorized_result(j):
    e = np.zeros((10, 1))
    e[j] = 1.0
    return e
```

Збережіть та закрийте файл `mnist_loader.py`.

Запуск програми

У середовищі розробки IDLE послідовно виконайте наступні команди для встановлення робочого каталогу на прикладі `C:\NeuralNetwork`:

```
>>>import os  
>>>os.chdir ('C:\\NeuralNetwork\\Network1')
```

Наступні команди використовуються для підключення модуля `mnist_loader` та ініціалізації наборів даних для навчання нейронної мережі:

```
>>>import mnist_loader  
>>>training_data, validation_data, test_data =  
mnist_loader.load_data_wrapper()
```

Підключіть створений вами модуль `network.py`:

```
>>>import network
```

При цьому виконається написана в ньому програма, що виводить інформацію про нейронну мережу.

Створіть нейронну мережу для розпізнавання рукописних цифр:

```
>>>net = network.Network([784, 30, 10])
```

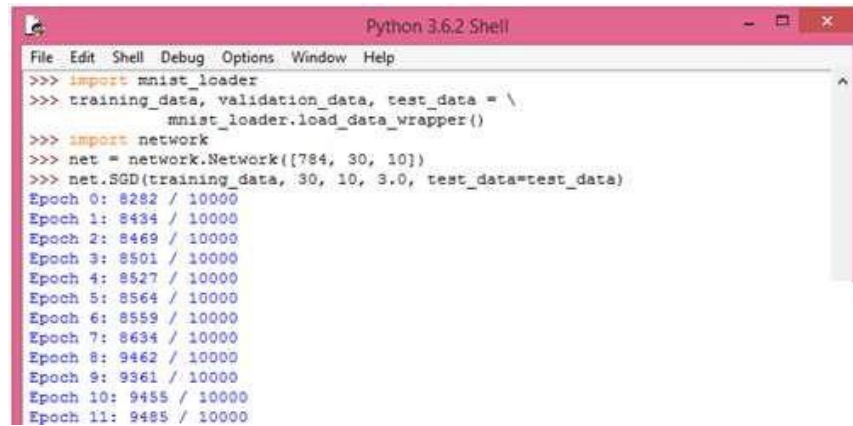
Параметри, вказані під час виклику даного методу, визначають топологію мережі, що створюється. Таким чином, в результаті виконання команди буде створено мережу, що складається з трьох шарів: вхідний шар мережі складається з 784 нейронів; внутрішній шар з 30 нейронів та вихідний шар з 10 нейронів.

Запустіть процедуру навчання створеної нейронної мережі, що включає 30 епох:

```
>>>net.SGD(training_data, 30, 10, 3.0, test_data=test_data)
```

Параметри, вказані під час виклику методу `SGD`: навчальна вибірка, кількість епох навчання, розмір підвибірки, швидкість навчання, вибірка, що тестує. Навчання може тривати кілька хвилин. Під час навчання видаватиметься інформація про пройдені епохи (див. рис. 1). Для кожної епохи виводиться відношення кількості правильно розпізнаних цифр до загальної кількості цифр тестової вибірки.

Наприклад, запис `Epoch 6: 9374 / 10000` говорить про те, що в результаті епохи навчання з номером 6 досягнута точність розпізнавання ≈ 0.94 , що складає 94%.



```

Python 3.6.2 Shell
File Edit Shell Debug Options Window Help
>>> import mnist_loader
>>> training_data, validation_data, test_data = \
mnist_loader.load_data_wrapper()
>>> import network
>>> net = network.Network([784, 30, 10])
>>> net.SGD(training_data, 30, 10, 3.0, test_data=test_data)
Epoch 0: 8282 / 10000
Epoch 1: 8434 / 10000
Epoch 2: 8469 / 10000
Epoch 3: 8501 / 10000
Epoch 4: 8527 / 10000
Epoch 5: 8564 / 10000
Epoch 6: 8559 / 10000
Epoch 7: 8634 / 10000
Epoch 8: 9462 / 10000
Epoch 9: 9361 / 10000
Epoch 10: 9455 / 10000
Epoch 11: 9485 / 10000

```

Рис.1. Результат роботи програми Network1.

Завдання:

1. Вивчіть алгоритм та роботу програми Network1. Введіть код розглянутого прикладу та отримайте результат роботи програми Network1.
2. Ознайомтеся самостійно з бібліотекою **Fashion MNIST** (її можна завантажити за посиланням <https://www.kaggle.com/zalando-research/fashionmnist> або знайдіть інший спосіб її завантаження).
3. Створіть аналогічну програму **Network2** для розпізнавання набору даних **Fashion MNIST** з використанням активаційної функції **ReLU** (подивіться документацію Python). Змінюючи гіперпараметри, домагайтеся мінімізації помилки розпізнавання. (Зауваження. При використанні функції ReLU треба уважно віднестись не тільки до правильного введення цієї функції та її похідної, але й до тих даних, які ми отримуємо на виході нашої нейронної мережі для реалізації SGD, тобто треба буде переробити `backprop`).
4. Після завершення навчання мережі, продемонструйте її роботу на *двох будь яких довільно вибраних прикладах зображень бібліотеки Fashion MNIST*. Для цього створіть програму рандомного вибору прикладів з бібліотеки та подайте їх на вхід мережі, отримайте результат класифікації. Також зробіть процедуру візуалізації цих двох обраних зображень.
5. Порівняйте результати роботи двох мереж, зробіть висновки щодо якості їх роботи з різними активаційними функціями.

Зміст звіту:

- Коди програми **Network2** із коментарями;
- Скріншоти з результатами навчання створеної мережі для бібліотеки **Fashion MNIST** та з результатами класифікації вибраних прикладів;
- Висновки.